

Chapter 1 · Lesson 2 — Data Modeling (TypeScript interfaces)

Chapter 1 · Lesson 2 — Data Modeling (TypeScript interfaces)

Goal: translate yesterday's pub conversation into precise TypeScript types. By the end of this lesson you'll have written `Lib/types.ts` — the single most important file in the codebase, even though it doesn't render anything.

Why we model the data before we render it

There's a saying in engineering: **"Show me your data structures and I'll know your code."** It's attributed to Linus Torvalds, but the idea is older. The shape of your data **dictates the shape of your code**. Everything else is decoration.

If you walk into a React interview and the interviewer hands you a problem, the first move that separates seniors from juniors is *what you draw on the whiteboard first*. Juniors draw a component tree. Seniors draw the data. Once the data is right, the component tree falls out almost mechanically.

So before we ever import React, we're going to spend this lesson drawing the data — but in TypeScript instead of on a whiteboard.

A 60-second TypeScript primer (for total beginners)

If you've never used TypeScript, two things to know:

1. **TypeScript is JavaScript plus type annotations.** Every TS file is also a valid JS file once the types are erased. Types are a development-time feature; they vanish at runtime.

2. **Types describe shapes.** When we write `interface Match { id: number; winner?: string; }`, we are saying “any object claiming to be a *Match* must have an *id* field that is a number, and may optionally have a *winner* field that is a string.”

That’s almost all you need for this lesson. Two more details, then we’ll write some types:

- `?` after a field name means **optional** — the field may be present or absent. `winner?: string` means *winner might be a string, or might not exist at all (undefined)*.
- `|` is a **union** — “either this or that”. `pick: string | null` means *pick is either a string or the literal value null*.

That’s the whole TypeScript story we need for Chapter 1. Don’t overthink it. The compiler is just a really pedantic friend reading over your shoulder.

Modeling Match

A match is a single contest between two players. From Lesson 1 we know it has:

- A position in the bracket (its “id”, really an ordinal — match 1, match 2, ...)
- Two players (let’s call them p1 and p2)
- A winner — *but only after the match is played*
- A score — display only, like ‘10-7’
- For Round 1, the seed of player 1 (because R1 pairs are usually shown as “1 vs 32”, “2 vs 31”, etc.)

In TypeScript:

```
export interface Match {  
  id: number;  
  p1: string;  
  p2: string;  
  winner?: string;  
  score?: string;  
  seed1?: number;  
}
```

Six fields, three required, three optional. Read each `?` carefully — the `?` is **load-bearing**. It’s the difference between “*this field always exists*” and “*this field exists only sometimes*.”

Why winner? is optional

This is the fork in the road that powers the whole app. Re-read Lesson 1’s scoring rules: a match that hasn’t been played has `null` (or, equivalently here, `undefined`) for every team’s score. The data-level mechanism that produces that behaviour is **the absence of `match.winner`**.

When the file owner adds a winner, the field appears. Until then, it’s not there at all. There is no winner: `null` in the data, no winner: `'`, no winner: `'TBD'`. There is *just no winner field*. That’s the cleanest way to model “not yet known”: **the field doesn’t exist**.

The TypeScript compiler will then, on its own, force every consumer of `match.winner` to handle the undefined case:

```
const w = match.winner;           // type is `string | undefined`
const isWin = pick === w;         // works, but isWin is false if w is undefined
if (match.winner) { /* ... */ }  // narrowed to `string` inside this block
```

That’s the type system **doing free safety work for you**. If you’d modeled winner as `winner: string` (required), every place that called `scorePick` would happily compare `pick === ''` and silently return *consolation* instead of *pending*. The optional `?` is what saves you.

Why score? and seed1? are optional

`score` is optional because finished matches have one and pending matches don’t. Same logic as `winner`.

`seed1` is more interesting. It’s only on R1 matches in our data. Why? Because that’s the only round where the *bracket position is meaningful enough to display*. By round 2, “seed 1 vs unseeded qualifier” is no longer the point — the point is the matchup itself, “Zhao vs Murphy”. So we never bother to add seeds in R2/QF/SF/F objects, and the type accommodates that with `?`. Optional fields aren’t about “data we forgot to fill in” — they’re about **fields that simply don’t exist for this row**.

Modeling Team

A team has a name, some cosmetics (color, accent, icon, motto), and **five arrays of picks**:

```

export interface Team {
  name: string;
  color: string;
  accent: string;
  icon: string;
  motto: string;
  r1: string[];
  r2: string[];
  qf: string[];
  sf: string[];
  final: string;
  // Round-2 lookup tables for handling players whose path traveled through a
  // bracket halves than the simple seed pairing implies (used by scoring as
  r1Players?: string[];
  r2Players?: string[];
}

```

Three things to notice:

1. **r1, r2, qf, sf are arrays of strings — but final is just a string.**

Lesson 1 reminded us there's only one Final match. So a team only makes one Final pick. We *could* have written `final: string[]` with one element, but that would cost us readability everywhere we touch it: `team.final[0] === 'Wu Yize'` reads worse than `team.final === 'Wu Yize'`. So we accept the asymmetry. The cost is that scoring code has one branch for Final and another for everything else (we'll see this in Lesson 3).

Asymmetry in your data is fine when it matches the real-world asymmetry. “There is one final” is a real-world constraint. The data should reflect it.

2. **The cosmetics live next to the picks.**

color, accent, icon, motto are decorative. They drive every gradient, badge, and chip in the UI. They live on the same object as the picks because **each team is one logical thing** — not two (a “team identity” plus “team picks”). Splitting them would buy nothing and add a level of indirection.

This is a common beginner mistake: over-normalizing. SQL databases want you to factor

out shared data into separate tables. **A 250-row JavaScript file does not.** When the data is small and self-contained, denormalize. Optimize for reading the data with your eyes, not for storage efficiency.

3. `r1Players?` and `r2Players?` exist for a subtle scoring case.

These aren't needed for normal scoring — they're a hint used by `calculateTeamScores` when a team's pick *advances* through the round in a non-obvious way (e.g. the team picked someone in their R1 spot, but the R2 spot is for a different bracket position than they expected). For most teams these arrays are unused. They're optional precisely because most teams won't have them.

We'll come back to this in Lesson 3 when we write the scoring function. For now, just know the field exists and is rare.

The most important rule in the codebase (the invariant)

This is the thing that, if you forget it, every score in the app silently goes wrong:

For each round, the *i*-th element of `team.r1` (or `team.r2`, `qf`, `sf`) corresponds to the *i*-th element of `ROUND1_MATCHES` (or `ROUND2_MATCHES`, `QF_MATCHES`, `SF_MATCHES`).

There is no `matchId` field on each pick. There is no key linking a pick to a match. **Order is the contract.** Index 0 of `team.r1` is the team's pick for `ROUND1_MATCHES[0]`. Index 5 is the pick for `ROUND1_MATCHES[5]`. Always.

If you re-order `ROUND1_MATCHES` and forget to re-order all 8 teams' `r1` arrays in lockstep, every Round 1 score is wrong. There is **no warning**, no compile error, no runtime test catching this. It's a convention enforced by you, the developer.

In a "proper" enterprise codebase you'd add a foreign key — each pick would be { `matchId: 5`, `player: 'Mark Allen'` } and scoring would `find()` the match by id. That's safer but **noisier to read and write**. For an 8-team league with one owner editing the file, the convention is fine. The moment a 9th team or 33rd player enters, you'd reconsider.

This is a recurring tradeoff in software: **safety vs. simplicity**. Senior engineers know when each is right. The answer is rarely "always one or always the other."

Modeling PlayerInfo (the dictionary, not an array)

There are 32 players. Each has a country flag, a seed, a status string, a flag-text label:

```
export interface PlayerInfo {  
  country: string;    // emoji flag, e.g. '🇨🇳'  
  seed?: number;      // 1-16, or undefined for qualifiers  
  status: string;     // 'Defending Champion', 'World No. 1', etc.  
  flag: string;       // 3-letter, e.g. 'CHN'  
}
```

Notice `seed?` is optional — only the top 16 are seeded. Qualifiers don't have a seed.

But the more interesting decision is what *container* we put 32 of these in. Two options:

```
// Option A: array of objects with a 'name' field  
export const PLAYERS: { name: string; country: string; seed?: number; ... }[] = [  
  { name: 'Zhao Xintong', country: '🇨🇳', seed: 1, ... },  
  // ...  
];  
  
// Option B: object keyed by name  
export const PLAYER_INFO: Record<string, PlayerInfo> = {  
  'Zhao Xintong': { country: '🇨🇳', seed: 1, ... },  
  // ...  
};
```

Both are valid. The codebase picks **Option B**. Why?

Because the most common operation in the app is “given a player’s name (from a match’s *p1/p2*, or from a team’s pick), get the metadata for that player.” That’s a hash lookup:

```
const info = PLAYER_INFO['Zhao Xintong']; // O(1)
```

If we’d picked Option A, every lookup would be:

```
const info = PLAYERS.find(p => p.name === 'Zhao Xintong'); // O(n)
```

For 32 players, the performance difference is negligible. The **readability** difference, however, is significant — and the readability difference is multiplied by the number of times the lookup appears in the code. In our codebase, it appears in `<PlayerDetail>`, in

<MatchPickAnalysis>, in the standings hover tooltip, in the player card, etc. Probably 10+ places. Each of them reads cleaner with Option B.

This decision — *what's the lookup key?* — is one you'll make on every dataset. Ask yourself *"how will I most commonly query this?"* The answer becomes the key.

Modeling the score detail (output of scoring, used by every UI)

When `calculateTeamScores` runs, it doesn't just return a number. It returns **detail records** that the UI uses to color cells, write tooltips, count corrects/wrongs. Each detail looks like this:

```
export interface ScoreDetail {
  match: Match;
  pick: string;
  points: number | null;    // 3 | 1 | null
  correct: boolean | null;  // true | false | null
}
```

Two tri-state fields. `points` is 3 (correct), 1 (wrong), or null (pending). `correct` is true, false, or null (pending).

You might ask *"why both?"* Aren't correct and points redundant? Almost, but not quite:

- `points` is what you sum to get a total.
- `correct` is what you count to get a hit-rate (e.g. *"Invincibles got 11 of 16 R1 picks right"*).

Both come up. The UI reads `correct` for stat tiles ("11 of 16") and `points` for the running totals ("33 / 48"). Storing them both, computed once, saves every UI place from re-deriving one from the other.

Putting it all together: `lib/types.ts`

Here's the whole file, all in one place. Open `lib/types.ts` in the project root and compare:

```
// lib/types.ts

export interface Match {
```

```

    id: number;
    p1: string;
    p2: string;
    winner?: string;
    score?: string;
    seed1?: number;
}

export interface PlayerInfo {
    country: string;
    seed?: number;
    status: string;
    flag: string;
}

export interface ScoreDetail {
    match: Match;
    pick: string;
    points: number | null;
    correct: boolean | null;
}

export interface TeamScores {
    r1: number;
    r2: number;
    qf: number;
    sf: number;
    f: number;
    total: number;
    r1Details: ScoreDetail[];
    r2Details: ScoreDetail[];
    qfDetails: ScoreDetail[];
    sfDetails: ScoreDetail[];
    fDetails: ScoreDetail[];
}

```



```

export interface Team {
  name: string;
  color: string;
  accent: string;
  icon: string;
  motto: string;
  r1: string[];
  r2: string[];
  qf: string[];
  sf: string[];
  final: string;
  r1Players?: string[];
  r2Players?: string[];
}

export interface TeamWithScores extends Team {
  scores: TeamScores;
}

```

Six interfaces. Read each one. **None of them imports React.** None has a JSX angle bracket. This is pure data modeling, and it's the most important file in the project.

The `extends Team` at the bottom is a TypeScript trick: a `TeamWithScores` is a `Team` with one extra field, `scores: TeamScores`. The orchestrator component (Chapter 4) builds `TeamWithScores` objects by spreading the original team and adding scores. Until then, just notice that the type system tracks both.

Why the TypeScript compiler is your friend

If you typed `team.r1[0]` somewhere in this app, TypeScript knows it's a `string`. If you typed `match.winner` somewhere, TypeScript knows it might be undefined and forces you to handle that. If you tried to assign a `Team` to a variable typed `Match`, TypeScript yells at you immediately.

The types are the contract. Once they're written, every piece of code that touches a `Team` or `Match` has to obey them. That's the safety net that lets you refactor confidently — change a type, and the compiler tells you every file that breaks.

A common interview question: “*why use TypeScript instead of plain JavaScript?*” The answer in two parts: (1) **catch errors at compile time** instead of runtime, (2) **document the data shapes** so future-you (or future-collaborator) knows what’s going where. The second one is at least as important as the first. Types are documentation that **doesn’t drift out of sync with the code**, because the code won’t compile if it does.

The quick exercise

Open `lib/types.ts` in the repo. Compare it to what we wrote above. Is there anything different? (Spoiler: no — the file is exactly what’s in this lesson, possibly with the order reshuffled. The types in the lesson are the live, working types in the app.)

If you’re following along by re-creating the project from scratch, type the file out yourself. Don’t copy-paste. Typing it is how the patterns sink in. The whole file is about 50 lines.

Vibe prompt you would have used

“Write me a `lib/types.ts` file with TypeScript interfaces for: (1) `Match` with `id: number`, `p1: string`, `p2: string`, `optional winner?: string`, `optional score?: string`, `optional seed1?: number`; (2) `PlayerInfo` with `country`, `status`, `flag` strings and `optional seed?: number`; (3) `Score-Detail` with `match: Match`, `pick: string`, `points: number | null`, `correct: boolean | null`; (4) `TeamScores` with the 5 round totals plus `total` and 5 `Details` arrays; (5) `Team` with `name`, four cosmetic strings, picks arrays for `r1[]`, `r2[]`, `qf[]`, `sf[]`, a single `final: string`, and two optional helper arrays; (6) `TeamWithScores` extends `Team` with one additional `scores: TeamScores` field. Don’t write any other code.”

Notice how specific every field name is. This is what a good vibe prompt looks like for type generation: each field is named, each type is given, each `?` is intentional. The LLM has nothing to hallucinate. **Specificity in the prompt means specificity in the output.**

CHECK YOURSELF

1. **What’s the difference between a `winner?: string` and `winner: string | undefined` in TypeScript?** (Subtle — answer at the end of this lesson.)

2. `team.final` is a `string` but `team.sf` is a `string[]`. Justify the asymmetry. What would it cost in code-readability to “fix” it by making `final` an array of one?
3. `PLAYER_INFO` is an object keyed by name, not an array. Name two other places in the app where this lookup pattern would be cheaper than `.find()`. (You don’t have to know the codebase yet — guess.)
4. The “match arrays in pick array order” invariant is enforced by convention, not by the type system. What change to the `Team` interface would let TypeScript enforce it for you? What’s the cost?
5. Open `lib/types.ts` and find one type you don’t immediately understand. Read it three times. Write down what every field is for. (No question, just an exercise.)

Answer to #1 (because it’s a real gotcha)

`winner?: string` and `winner: string | undefined` look almost the same, but they differ on **whether the field has to be specified at all** when constructing the object.

```
interface A { winner?: string; }
interface B { winner: string | undefined; }

const a: A = {}; // ? valid (field omitted)
const a2: A = { winner: undefined }; // ? valid (field present, undefined)

const b: B = {}; // ? ERROR: 'winner' is missing
const b2: B = { winner: undefined }; // ? valid (field present, undefined)
```

`?` says “the property may not exist on the object at all.” `undefined` says “the property must exist, but its value can be undefined.” For our `Match` type we want `?` — pending matches simply don’t have a `winner` field, period.

When you’ve got these answered, head to [03-pure-functions-and-scoring.md](#).